Lab Assignment: Reinforcement learning

Lab team: 12

Name (member 1): Lucas Miranda López

Name (member 2): Iván Domínguez Hernández

- Please include your full name at the beginning of all submitted files.
- Make sure the presentation is well-structured: the report will be evaluated not only for correctness, but also for clarity, conciseness, and completeness.
- Make use of figures and tables to summarize the results and illustrate the discussions.
- If external material is used, the sources must be cited.
- Include references in APA format https://pitt.libguides.com/citationhelp/apa7. Lack or poorly formatted references can be penalized.
- A generative AI tool can be used for consultation. You must specify the tool used in your report.
- You are not allowed to use a generative AI tool to generate code.

Submit a single `.zip` file, whose name has the format `AA3_P04_teamCode_lastName1_lastName2.zip` The name must not include graphical accents, spaces, uppercase letters, or special characters.

For example: `AA3_P04_G03_munyoz_deLaRosa.zip`

This compressed file must include the following files:

- This Python notebook with the solutions of the exercises. The notebook should include only code snippets, figures, tables, derviations and explanations (with LaTex if necessary) in Markdown cells. Handwritten material can be included in the Python notebook as images. Functions should be defined in a separate `.py` file, not in the notebook.
- The necessary `.py` and addional files to ensure the Python notebook code can be executed sequentially without errors.
- A PDF file generated from the notebook (Export the notebook as an HTML file. Open the HTML file in a Browser and print it as a PDF file).

Make sure that all the code cells can be executed squentially without errors (Kernel -> Restart & Run All). Exectution and formatting errors will be penalized.

Evaluation criteria:

- [6 points] Quality of the report (correctness, clarity, conciseness, completeness).
- [3 points] Quality of the code (correctness, adherence to a Python style guide -for instance, Google's-, comments, functional decomposition).
- [1 point] References.

Training a reinforcement learning agent at the gymnasium 🚀

📚 RL-environment:

- Python and NumPy
- Gymnasium

🎮 Environment:

- FrozenLake-v1

📚 Pygame:

https://www.pygame.org/wiki/about

Exercise 1: The gymnasium

TO DO: Complete the gymnasium tutorials

- Basic usage: https://gymnasium.farama.org/introduction/basic_usage/
- Training an RL-agent: https://gymnasium.farama.org/introduction/train_agent/

In [32]:
```
%load_ext autoreload
%autoreload 2

import numpy as np
import matplotlib.pyplot as plt
import gymnasium as gym
import imageio
import os

from tqdm.notebook import tqdm
import rl_utils
import time
from IPython.display import display, clear_output


from reinforcement_learning import (
    sarsa_learning,
    q_learning,
    greedy_policy,
    epsilon_greedy_policy,
)
```

```
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
```

In [33]:
```
# Install packages if needed
# !pip install pygame
# !pip install gymnasium
```

Exercise 2: Q-learning

We're now ready to code our Q-Learning algorithm 🔥

Step 0: Set up and understand Frozen Lake environment

Let's begin with a simple 4x4 map and non-slippery, meaning the agent always moves in the desired direction.

We add a parameter called `render_mode` that specifies how the environment should be visualised. In our case because we **want to record a video of the environment at the end, we need to set render_mode to rgb_array**.

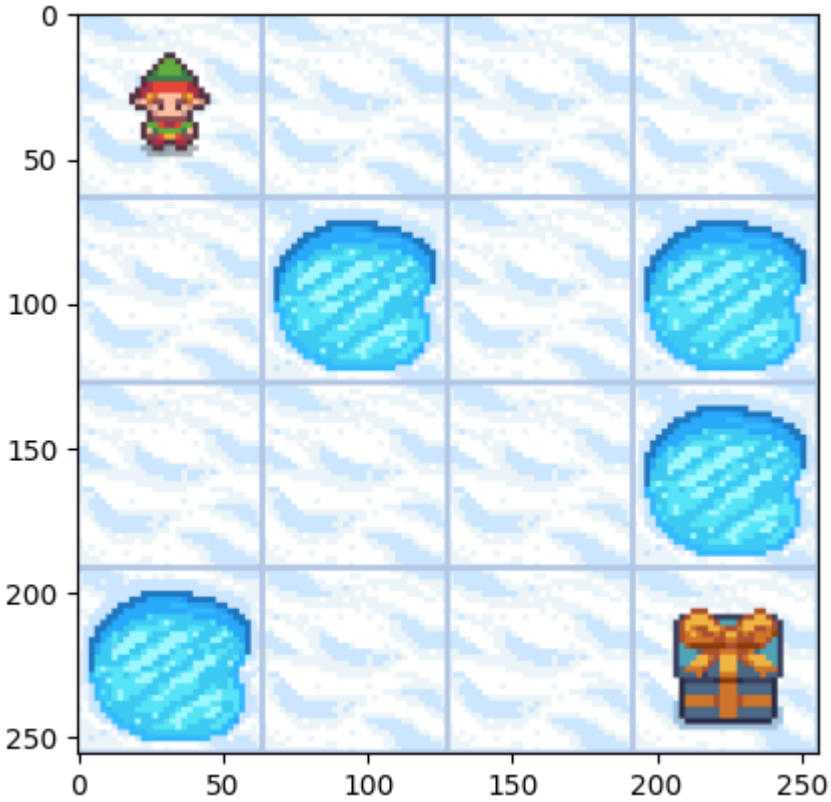
As explained in the documentation "rgb_array": Return a single frame representing the current state of the environment. A frame is a np.ndarray with shape (H, W, 3) representing RGB values for an H (height) times W (width) pixel image.

In [34]:
```
small_environment = gym.make(
    'FrozenLake-v1',
    map_name='4x4',
    is_slippery=False,
    render_mode='rgb_array',
)
```

Let's see what the environment looks like:

In [35]:
```
state, info = small_environment.reset()  # observation state
action_names = {0: 'Left', 1: 'Down', 2: 'Right', 3: 'Up'}

print(state)
print(info, '(Probability that the action has led to the current state)')
n_states = small_environment.observation_space.n
print("There are ", n_states, " possible states")

n_actions = small_environment.action_space.n
print("There are ", n_actions, " possible actions")
print(action_names)

fig, ax = plt.subplots()
game_image = ax.imshow(small_environment.render())
```

```
0
{'prob': 1} (Probability that the action has led to the current state)
There are  16  possible states
There are  4  possible actions
{0: 'Left', 1: 'Down', 2: 'Right', 3: 'Up'}
```



In [36]:
```
# Generate a random observed state
print("Observation state (randomly selected)", small_environment.observat

# Generate a random action from the current state
print("Action (randomly selected):", small_environment.action_space.sampl
```

```
Observation state (randomly selected) 11
Action (randomly selected): 2
```

In [37]:
```
#  Generate an episode
obsevation, info = small_environment.reset()  # state state_state
fig, ax = plt.subplots()
game_image = ax.imshow(small_environment.render())

MAX_STEPS = 100
refresh_rate = 1 # in (1 / seconds)

episode_over = False
n_steps = 0

while not episode_over and n_steps < MAX_STEPS:
    n_steps += 1
    action = small_environment.action_space.sample()

    state, reward, terminated, truncated, info = small_environment.step(a
    episode_over = terminated or truncated

    ax.set_title(
        'Step: {}  State: {}  Reward: {} Action:{}'.format(
            n_steps,
            state,
            reward,
            action_names[action],
        )
    )
```

```
    display(fig)
    time.sleep(1.0 / refresh_rate)
    clear_output(wait=True)  # Clear previous output
    game_image.set_data(small_environment.render())

small_environment.close()
```

Step 1: Greedy and Epsilon greedy policies

Since Q-Learning is an **off-policy** algorithm, we have two policies. This means we're using a **different policy for acting and updating the value function**.

- Epsilon-greedy policy (acting policy)
- Greedy-policy (updating policy)

The greedy policy will also be the final policy we'll have when the Q-learning agent completes training. The greedy policy is used to select an action using the Q-table.

Epsilon-greedy is the training policy that handles the exploration/exploitation trade-off.

- With *probability 1-ε* : **we do exploitation** (i.e. our agent selects the action with the highest state-action pair value).
- With *probability ε*: we do **exploration** (trying a random action).

As the training continues, we progressively **reduce the epsilon value since we will need less and less exploration and more exploitation.**

TO DO: Implement these policies in `reinforcement_learning.py`

Step 2: Train the RL agent 🏃

TO DO: Implement the Q-learning algorithm in `reinforcement_learning.py`

Define the hyperparameters for the learning process ⚙️

The exploration related hyperparamters are some of the most important ones.

- We need to make sure that our agent **explores enough of the state space** to learn a good value approximation. To do that, we need to have progressive decay of the epsilon.
- If you decrease epsilon too fast (too high decay_rate), **you take the risk that your agent will be stuck** in a local optimum, since your agent didn't explore enough of the state space and hence can't solve the problem.

The specific values chosen for the 4×4 non-slippery environment are justified as follows.

- **n_training_episodes = 5 000**: The 4×4 map has $|\mathcal{S}| \times |\mathcal{A}| = 16 \times 4 = 64$ state-action pairs. Following the coupon collector argument, covering all of them under random exploration requires $O(64 \log 64) \approx 266$ steps — roughly 27 episodes at an average step count of 10 during the early exploration phase. A budget of 5 000 episodes is well above what is needed to visit all pairs many times over, ensuring reliable convergence (Watkins & Dayan, 1992).

- **learning_rate = 0.7**: The environment is deterministic, so each transition carries no stochastic noise and a single Bellman update already gives an exact target. A large constant α accelerates convergence without introducing variance — exactly the setting where Sutton & Barto (2018, Section 6.1) recommend constant step sizes.

- **gamma = 0.95**: The shortest path to the goal in the 4×4 map takes $T^* = 6$ steps, so the discounted return along that trajectory is $\gamma^{T^*-1} = 0.95^5 \approx 0.77$. This means 77% of the reward signal reaches the initial state — sufficient for reliable credit assignment over the short horizons of this map.

- **decay_rate = 0.0005**: With $\lambda = 0.0005$, ε crosses below 0.5 at episode $t_{50} = \ln(19)/0.0005 \approx 5\,880 > 5\,000$, so it stays above 0.5 for the entire training run. The agent never commits to exploitation before the Q-table has been built — the exploration condition Singh et al. (2000) require for convergence.

References

Singh, S., Jaakkola, T., Littman, M. L., & Szepesvári, C. (2000). Convergence results for single-step on-policy reinforcement-learning algorithms. *Machine Learning, 38*(3), 287–308. https://doi.org/10.1023/A:1007678930559

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press. https://incompleteideas.net/book/the-book-2nd.html

Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning, 8*(3–4), 279–292. https://doi.org/10.1007/BF00992698

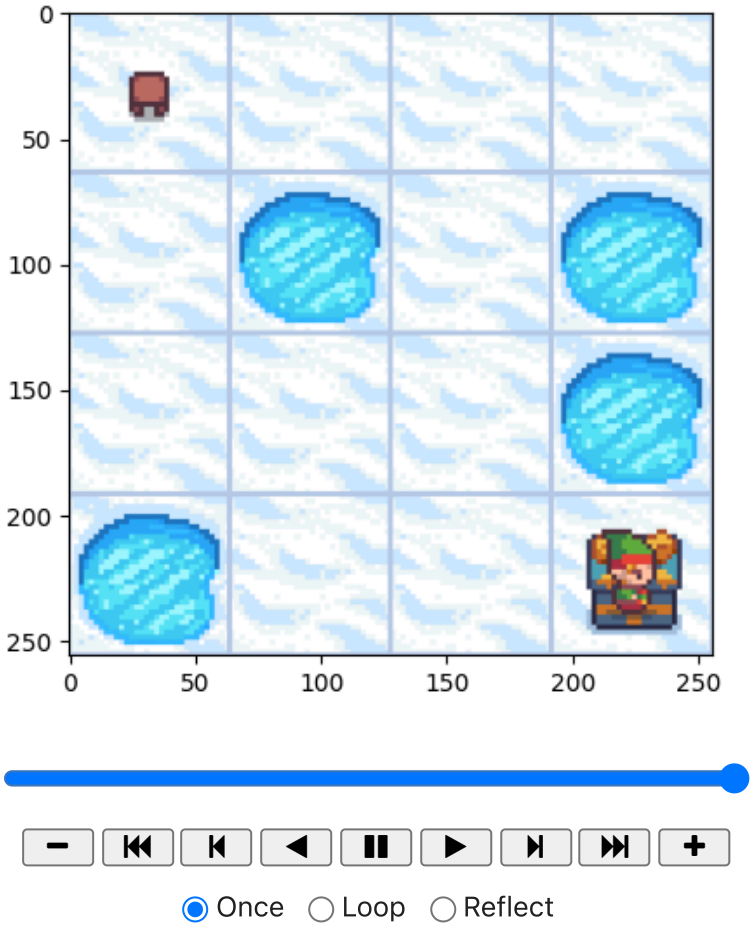
In [38]:
```
# Training hyperparameters

n_training_episodes = 5000
max_steps = 100 # Maximum number of steps per episode
learning_rate = 0.7
gamma = 0.95 # Discount factor

# Exploration parameters
max_epsilon = 1.0  # Initial exploration probability
min_epsilon = 0.05  # Minimum exploration probability
decay_rate = 0.0005 # Exponential decay rate for the exploration probabil
```

In [39]:
```
# Initialize Q-table
Qtable_small = np.zeros(
    (
        small_environment.observation_space.n,
        small_environment.action_space.n
    )
)

# Learn Q-table
Qtable_small, metrics_ql_small = q_learning(
    small_environment,
    n_training_episodes,
    max_steps,
    learning_rate,
    gamma,
    min_epsilon,
    max_epsilon,
    decay_rate,
    Qtable_small,
)
```

```
  0%|          | 0/5000 [00:00<?, ?it/s]
```

In [40]:
```
frames = rl_utils.generate_greedy_episode(small_environment, Qtable_small
rl_utils.show_episode(frames, interval=250)
```

Out[40]:



A more challenging problem

We're ready now to find our way in more challenging environments 💥

In [41]:
```
large_environment = gym.make(
    'FrozenLake-v1',
    map_name='8x8',
    is_slippery=False,
    render_mode='rgb_array',
    max_episode_steps=200,
)

n_states = large_environment.observation_space.n
print("There are ", n_states, " possible states")

n_actions = large_environment.action_space.n
print("There are ", n_actions, " possible actions")
```

```
There are  64  possible states
There are  4  possible actions
```

Clearly, those parameters won't be even close to solve the problem. So lets tune the hyperparameters one by one, as suggested in the literature:

- n_training_episodes = 10000

Q-learning is guaranteed to converge to Q^* only if every state-action pair (s, a) is visited infinitely often (Watkins & Dayan, 1992). In practice, a necessary condition is that the agent visits all

$$|\mathcal{S}| \times |\mathcal{A}| = 64 \times 4 = 256$$

pairs at least once. By analogy with the coupon collector problem applied to the $|\mathcal{S}| \times |\mathcal{A}| = 256$ state-action pairs, the expected number of transitions required to cover all pairs under uniform random exploration grows as $O(|\mathcal{S}| \cdot |\mathcal{A}| \cdot \log(|\mathcal{S}| \cdot |\mathcal{A}|))$. Since the 8×8 map has $\times 16$ more state-action pairs than the 4×4 map (256 vs. 64, for which 1 000 episodes sufficed), scaling to 10 000 episodes provides a conservative margin.

- gamma = 0.99

In FrozenLake, the only non-zero reward is $r = +1$ at the goal, received at step T^* of the optimal trajectory. The discounted return along that trajectory is:

$$G_0 = \gamma^{T^*-1} \cdot r_{T^*}$$

A standard measure of how far back the reward signal effectively propagates is the effective horizon:

$$H_\gamma = \frac{1}{1-\gamma}$$

For the 8×8 map, the length of the shortest path to the goal is approximately $T^* \approx 14$ steps. The table below compares the two candidate values:

γ	H_γ	γ^{14}
0.95	20	0.488
0.99	100	0.869

With $\gamma = 0.95$, more than half the reward signal is lost by the time it propagates back to the initial state. With $\gamma = 0.99$, 87% of the signal is preserved. As Sutton & Barto (2018, Section 3.3) note, choosing γ close to 1 is appropriate when the agent must plan over long horizons.

- decay_rate = 0.00001

The exploration probability decays as:

$$\varepsilon_t = \varepsilon_{\min} + (\varepsilon_{\max} - \varepsilon_{\min})\, e^{-\lambda t}$$

A practical design criterion is that ε_t should approach $\varepsilon_{\min}$ only after the agent has had sufficient time to cover the state space. Solving for the episode t_{90} at which ε has completed 90% of its decay from $\varepsilon_{\max}$ to $\varepsilon_{\min}$:

$$e^{-\lambda\, t_{90}} = 0.1 \implies t_{90} = \frac{\ln 10}{\lambda} \approx \frac{2.303}{\lambda}$$

With $\lambda = 0.00001$, this gives $t_{90} \approx 230\,000 \gg N$, meaning ε remains above 0.9 throughout all 10,000 training episodes. This guarantees the agent keeps exploring aggressively the whole time — a necessary condition when the 8×8 environment is sparse and the goal is hard to reach by chance, ensuring the Q-table is built reliably across different random seeds.

- learning_rate = 0.8

The Robbins–Monro conditions for asymptotic convergence of Q-learning require:

$$\sum_{t=0}^{\infty} \alpha_t = \infty \qquad \text{and} \qquad \sum_{t=0}^{\infty} \alpha_t^2 < \infty$$

A constant learning rate satisfies neither condition strictly, but in deterministic environments, where the transition $P(s' \mid s, a) \in \{0, 1\}$ carries no stochastic noise, each observed transition is exact. Consequently, a single Bellman update:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

is sufficient to correct any estimate, and a large constant α accelerates convergence without introducing variance. Sutton & Barto (2018, Section 6.1) explicitly recommend constant step sizes for deterministic or non-stationary problems.

References

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press. https://incompleteideas.net/book/the-book-2nd.html

Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning, 8*(3–4), 279–292. https://doi.org/10.1007/BF00992698

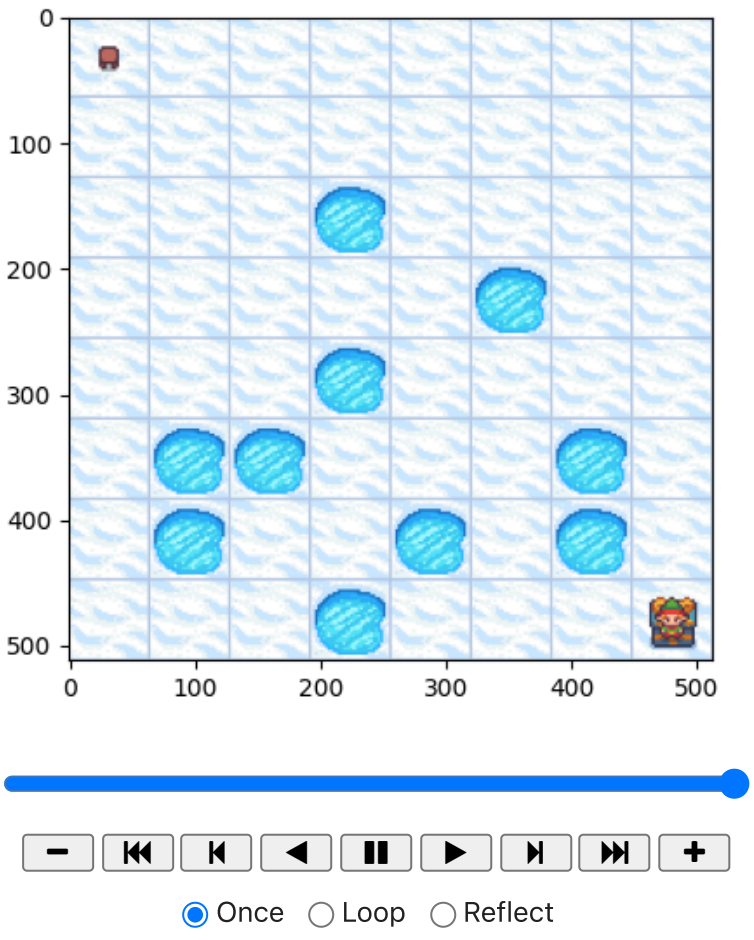
In [42]:
```
# Re-instantiate environment (generate_greedy_episode closes it)
large_environment = gym.make(
    'FrozenLake-v1',
    map_name='8x8',
    is_slippery=False,
    render_mode='rgb_array',
    max_episode_steps=200,
)

n_training_episodes = 10000
max_steps = 1000
learning_rate = 0.8
gamma = 0.99

# Exploration parameters
max_epsilon = 1.0
min_epsilon = 0.05
decay_rate = 0.00001

# Initialize Q-table
```

```
Qtable_large = np.zeros(
    (
        large_environment.observation_space.n,
        large_environment.action_space.n
    )
)

# Learn Q-table
Qtable_large, metrics_ql_large = q_learning(
    large_environment,
    n_training_episodes,
    max_steps,
    learning_rate,
    gamma,
    min_epsilon,
    max_epsilon,
    decay_rate,
    Qtable_large,
)
```

```
  0%|          | 0/10000 [00:00<?, ?it/s]
```

In [43]:
```
frames = rl_utils.generate_greedy_episode(large_environment, Qtable_large
rl_utils.show_episode(frames, interval=250)
```

Out[43]:



Slippery environment

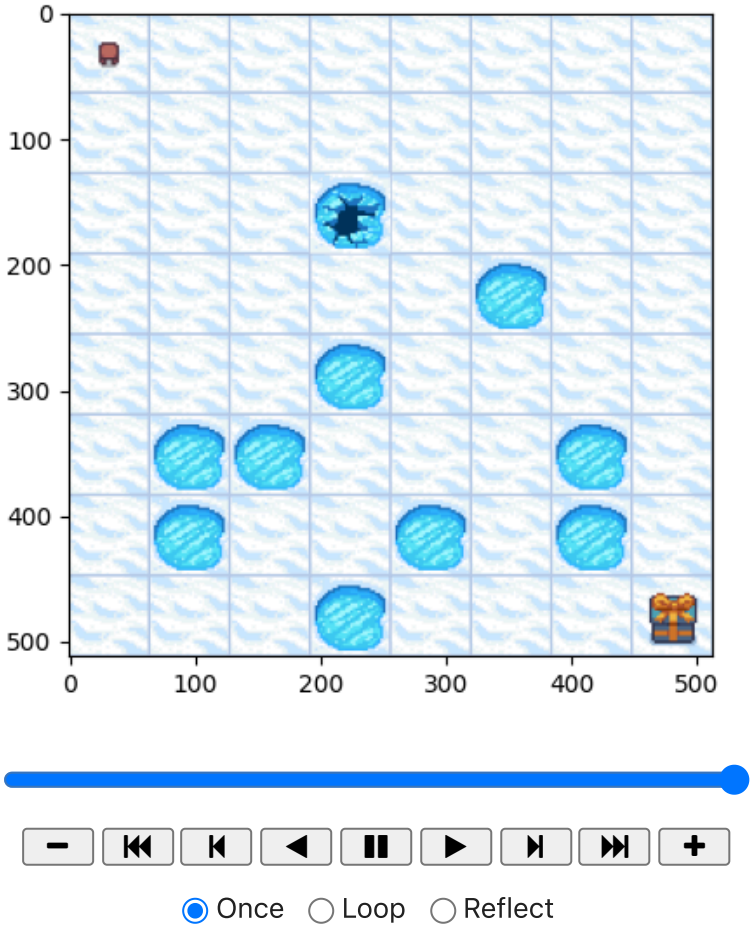
Our environment is now slippery, meaning the agent sometimes slips and moves in an unintended direction

In [44]:
```
slippery_environment = gym.make(
    'FrozenLake-v1',
    map_name='8x8',
    is_slippery=True,
    render_mode='rgb_array',
    max_episode_steps=200,
)
```

To visualize the challenges in this environment, let's make our agent go right several times and see what happens:

In [45]:
```
go_right = []
action = 2  # go-right
n_steps = 20
state, info = slippery_environment.reset()
for i in range(n_steps):
    go_right.append(slippery_environment.render())  # Capture current fra
    slippery_environment.step(action)

rl_utils.show_episode(go_right, interval=250)
```

Out[45]:



The slippery setting introduces stochastic transitions: with probability $1/3$ each, the agent moves in the intended direction or in one of the two perpendicular directions. Formally, the transition kernel becomes:

$$P(s' \mid s, a) = \frac{1}{3} \sum_{b \in \{a-1,\, a,\, a+1\} \pmod 4} \mathbf{1}[s' = T(s, b)]$$

where $T(s, b)$ is the deterministic next state for action b. This stochasticity has two consequences for hyperparameter selection.

- n_training_episodes = 50000

Stochastic transitions increase variance in the Q-table updates. Each (s, a) pair now maps to multiple possible next states, so a single visit is no longer sufficient to obtain a reliable estimate of $Q(s, a)$. The expected number of visits required for a stable estimate scales with the variance of the return, which in the slippery case is bounded by:

$$\mathrm{Var}[G_t] \leq \frac{R_{\max}^2}{(1-\gamma)^2}$$

Requiring five times as many episodes as the deterministic case (50 000 vs. 10 000) provides enough revisits to average out transition noise (Sutton & Barto, 2018, Section 6.1).

- learning_rate = 0.4

In a stochastic environment, each Bellman update carries noise proportional to the transition variance. Using a lower learning rate α reduces the step size and thus dampens the effect of individual noisy updates. Formally, the expected squared error of the update satisfies:

$$\mathbb{E}\left[(\delta_t)^2\right] = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q(s', a') - Q(s, a)\right)^2\right]$$

which is strictly larger in the stochastic case because s' is random. A smaller $\alpha = 0.4$ (down from the 0.8 used in the deterministic case) trades off convergence speed for stability, as discussed in Watkins & Dayan (1992).

- gamma = 0.99, max_epsilon = 1.0, min_epsilon = 0.05, decay_rate = 0.0001

The discount factor $\gamma = 0.99$ is unchanged: the optimal path length and the argument about signal propagation are the same as in the deterministic case. The decay rate, however, increases from $\lambda = 0.00001$ (deterministic) to $\lambda = 0.0001$ (slippery) for two reasons. First, with $N = 50\,000$ episodes the agent has five times more training time, so the cold-start problem is less acute: even with faster decay, ε remains above 0.5 for the first $\sim 7\,000$ episodes, providing sufficient random exploration to bootstrap the Q-table. Second, permanently high exploration is actively harmful for Q-learning in stochastic environments: the off-policy $\max_{a'} Q(s', a')$ target overestimates Q-values when both the transition s' and the Q-estimates are noisy, causing the table to oscillate rather than converge. With $\lambda = 0.0001$, $t_{90} = \ln(10)/\lambda \approx 23\,000$, so 90% of the decay is complete by episode

$23\,000$ — the first 46% of training — leaving the remaining 54% for near-greedy exploitation and policy consolidation.

References

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press. https://incompleteideas.net/book/the-book-2nd.html

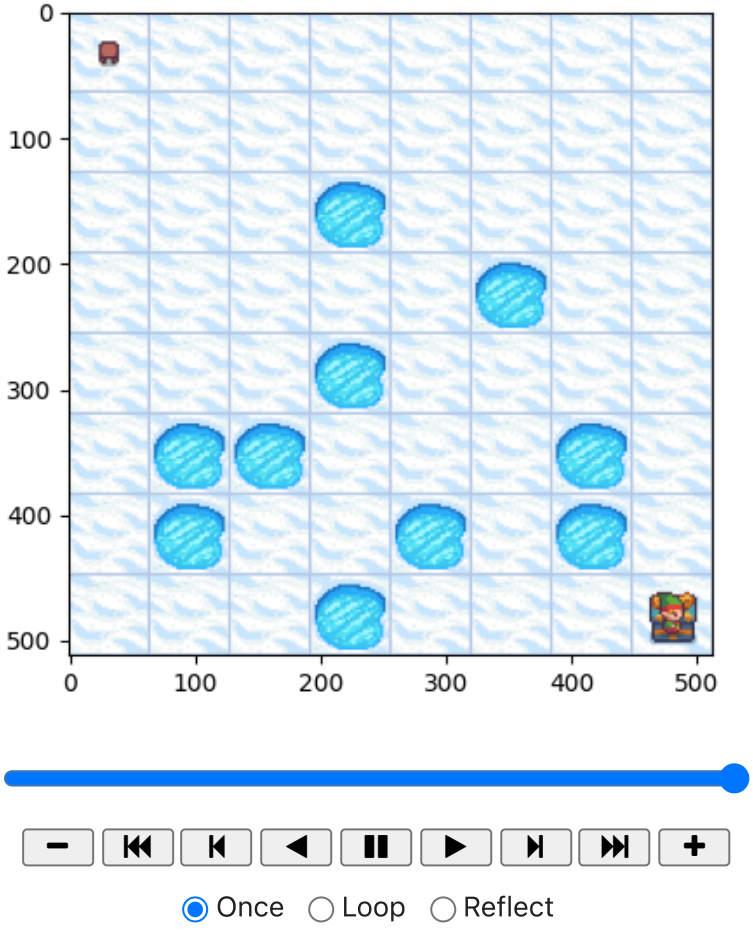
Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning, 8*(3–4), 279–292. https://doi.org/10.1007/BF00992698

In [46]:
```
# Re-instantiate environment
slippery_environment = gym.make(
    'FrozenLake-v1',
    map_name='8x8',
    is_slippery=True,
    render_mode='rgb_array',
    max_episode_steps=200,
)

n_training_episodes = 50000
max_steps = 200
learning_rate = 0.4
gamma = 0.99

# Exploration parameters
max_epsilon = 1.0
min_epsilon = 0.05
decay_rate = 0.0001

# Initialize Q-table
Qtable_slippery = np.zeros(
    (
        slippery_environment.observation_space.n,
        slippery_environment.action_space.n
    )
)

# Learn Q-table
Qtable_slippery, metrics_ql_slippery = q_learning(
    slippery_environment,
    n_training_episodes,
    max_steps,
    learning_rate,
    gamma,
    min_epsilon,
    max_epsilon,
    decay_rate,
    Qtable_slippery,
)
```

```
  0%|          | 0/50000 [00:00<?, ?it/s]
```

In [56]:
```
frames = rl_utils.generate_greedy_episode(slippery_environment, Qtable_sl
rl_utils.show_episode(frames, interval=250)
```

Out[56]:



Exercise 3: Train the RL agent using SARSA 🏃

TO DO: Implement the SARSA learning algorithm in `reinforcement_learning.py`

The SARSA hyperparameters are chosen by analogy with the Q-learning cases, with two systematic adjustments that reflect SARSA's on-policy nature.

- **4×4 non-slippery** — n_training_episodes = 5000, learning_rate = 0.7, gamma = 0.95, decay_rate = 0.001, max_steps = 100

The episode budget and discount factor mirror the Q-learning 4×4 baseline (5 000 episodes). The decay rate is increased slightly from $\lambda = 0.0005$ (Q-learning) to $\lambda = 0.001$ to ensure the policy keeps exploring long enough for the Q-table to receive meaningful reward signal. SARSA convergence requires the policy to be GLIE — Greedy in the Limit with Infinite Exploration — meaning ε must not decay too fast (Singh et al., 2000). A practical criterion is that ε should stay above 0.5 for at least the first half of training. Solving for the episode t_{50} at which ε crosses 0.5:

$$t_{50} = \frac{\ln\left(\frac{\varepsilon_{\max} - \varepsilon_{\min}}{0.5 - \varepsilon_{\min}}\right)}{\lambda} = \frac{\ln(19)}{\lambda}$$

With $\lambda = 0.005$ this gives $t_{50} \approx 590$ episodes — only 11.8% of the budget — so the policy becomes near-greedy while the Q-table is still essentially zero. Exploitation of an empty table then locks the agent into a fixed, likely suboptimal trajectory with no way to escape (Sutton & Barto, 2018, Section 2.2). With $\lambda = 0.001$, $t_{50} \approx 2\,940$ episodes (58.8% of the budget), leaving enough time to populate the Q-table before exploitation takes over. The learning rate $\alpha = 0.7$ is kept equal to the Q-learning baseline since the environment is deterministic and a constant step size is appropriate (Sutton & Barto, 2018, Section 6.1).

- **8×8 non-slippery** — n_training_episodes = 15000, learning_rate = 0.3, gamma = 0.99, decay_rate = 0.00001, max_steps = 200

The episode budget is increased to $15\,000$ (vs. $10\,000$ for Q-learning) and λ is set to 0.00001 for the same cold-start reason as the Q-learning 8×8 case: the sparse reward structure requires sustained exploration throughout all training episodes. The learning rate is reduced to $\alpha = 0.3$ (vs. 0.8 for Q-learning) because, with permanently high ε ($\varepsilon \approx 0.87$ at episode $15\,000$), SARSA's on-policy target $Q(s', a')$ is computed for actions drawn from an almost-random policy, introducing additional variance compared to Q-learning's deterministic $\max$ operator. A smaller α dampens this variance at the cost of slower convergence.

- **8×8 slippery** — n_training_episodes = 50000, learning_rate = 0.1, gamma = 0.99, decay_rate = 0.0001, max_steps = 200

The episode budget and λ match the slippery Q-learning case. The learning rate is further reduced to $\alpha = 0.1$ because SARSA in a stochastic environment faces compounded variance: the target $Q(s', a')$ is noisy both due to the random transition s' (stochastic environment) and the random action a' (epsilon-greedy policy). The expected squared TD error is therefore strictly larger than in the Q-learning case, and a smaller α is required to prevent the Q-table from oscillating. This is consistent with the recommendation in Sutton & Barto (2018, Section 6.4) that on-policy methods in stochastic environments benefit from more conservative step sizes.

References

Singh, S., Jaakkola, T., Littman, M. L., & Szepesvári, C. (2000). Convergence results for single-step on-policy reinforcement-learning algorithms. *Machine Learning, 38*(3), 287–308. https://doi.org/10.1023/A:1007678930559

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press. https://incompleteideas.net/book/the-book-2nd.html

In [57]:
```
# --- 4×4 non-slippery (same hyperparameters as Q-learning for fair compa
env_sarsa_small = gym.make('FrozenLake-v1', map_name='4x4', is_slippery=F

Qtable_sarsa_small, metrics_sarsa_small = sarsa_learning(
    env_sarsa_small,
    n_training_episodes=5000,
```

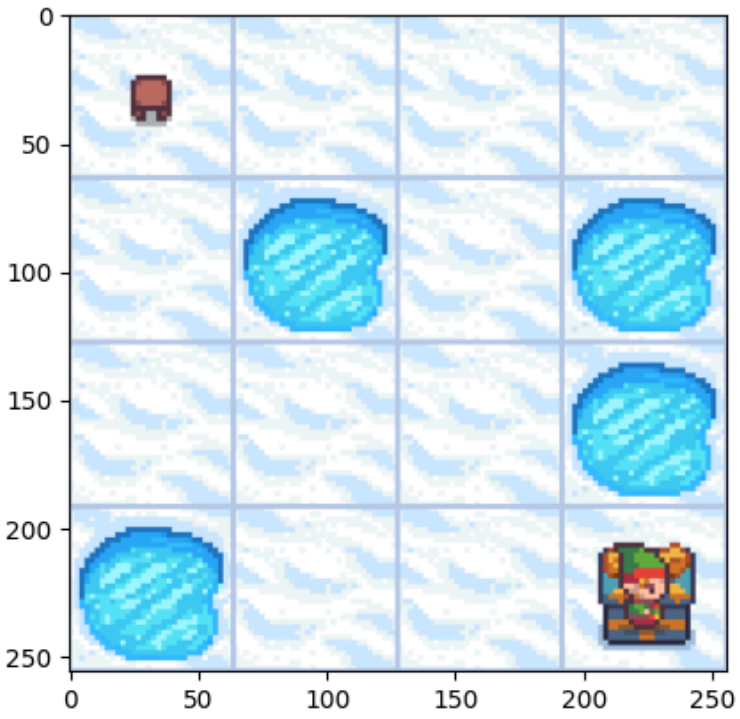
```
    learning_rate=0.7,
    gamma=0.95,
    min_epsilon=0.05,
    max_epsilon=1.0,
    decay_rate=0.001,
    max_steps=100,
    Qtable=np.zeros((env_sarsa_small.observation_space.n, env_sarsa_small
)

# --- 8×8 non-slippery ---
env_sarsa_large = gym.make('FrozenLake-v1', map_name='8x8', is_slippery=F

Qtable_sarsa_large, metrics_sarsa_large = sarsa_learning(
    env_sarsa_large,
    n_training_episodes=15000,
    learning_rate=0.3,
    gamma=0.99,
    min_epsilon=0.05,
    max_epsilon=1.0,
    decay_rate=0.00001,
    max_steps=200,
    Qtable=np.zeros((env_sarsa_large.observation_space.n, env_sarsa_large
)

# --- 8×8 slippery ---
env_sarsa_slippery = gym.make('FrozenLake-v1', map_name='8x8', is_slipper

Qtable_sarsa_slippery, metrics_sarsa_slippery = sarsa_learning(
    env_sarsa_slippery,
    n_training_episodes=50000,
    learning_rate=0.1,
    gamma=0.99,
    min_epsilon=0.05,
    max_epsilon=1.0,
    decay_rate=0.0001,
    max_steps=200,
    Qtable=np.zeros((env_sarsa_slippery.observation_space.n, env_sarsa_sl
)
```

```
  0%|          | 0/5000 [00:00<?, ?it/s]
  0%|          | 0/15000 [00:00<?, ?it/s]
  0%|          | 0/50000 [00:00<?, ?it/s]
```

In [65]:

```
for env, qtable, title in [
    (env_sarsa_small,    Qtable_sarsa_small,    '4×4 non-slippery'),
    (env_sarsa_large,    Qtable_sarsa_large,    '8×8 non-slippery'),
    (env_sarsa_slippery, Qtable_sarsa_slippery, '8×8 slippery'),
]:
    print(f'SARSA greedy episode — {title}')
    frames = rl_utils.generate_greedy_episode(env, qtable)
    display(rl_utils.show_episode(frames, interval=250))
```

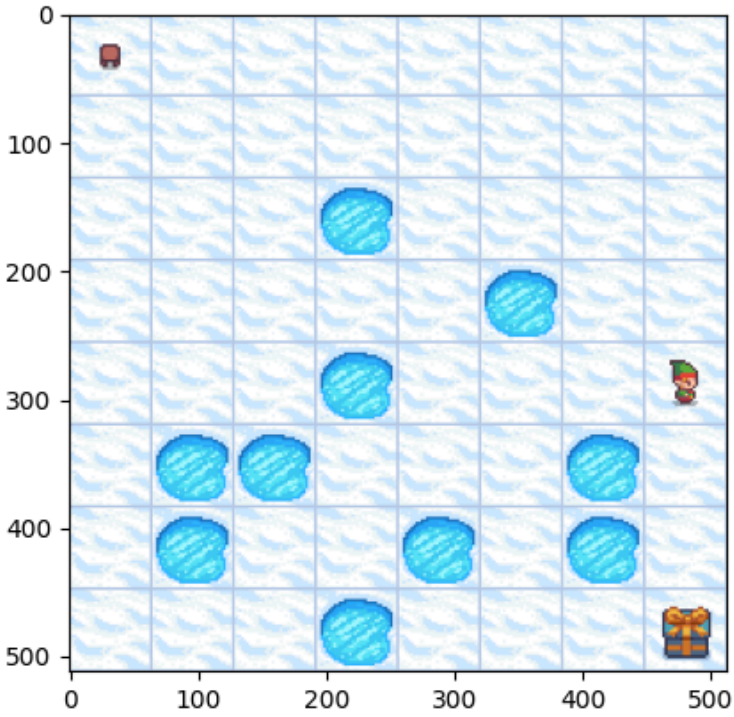
```
SARSA greedy episode — 4×4 non-slippery
```



Once Loop Reflect

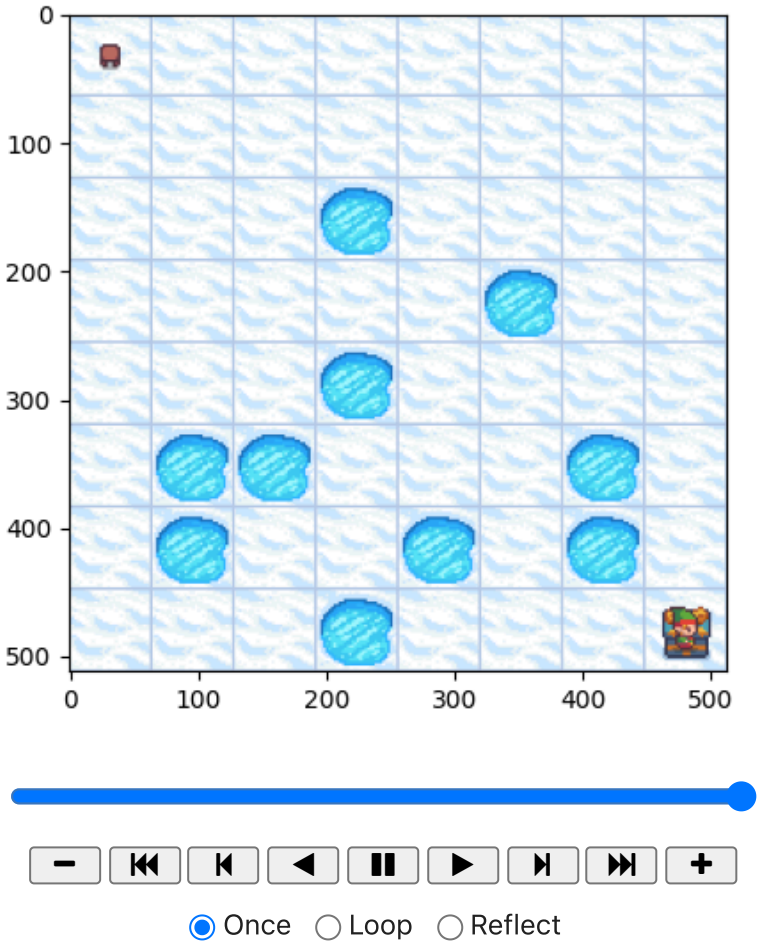
SARSA greedy episode — 8×8 non-slippery

```
Animation size has reached 21036809 bytes, exceeding the limit of 2097152
0.0. If you're sure you want a larger animation embedded, set the animatio
n.embed_limit rc parameter to a larger value (in MB). This and further fra
mes will be dropped.
```



Once Loop Reflect

SARSA greedy episode — 8×8 slippery



Exercise 4: Compare SARSA and Q-learning

Answer these questions:

- What is "episode reward" in RL and how is it related to the performance of an RL agent?
- What is "episode length" in RL and how is it related to the performance of an RL agent?
- What is "training error" in RL and how is it related to the performance of an RL agent?
  - Provide an explicit expression of the training error in the cases explored

Use matplotlib to compare the learning curves of SARSA and Q-learning, in terms of

- episode reward.
- episode length.
- training error
- Discuss the results obtained.

Conceptual framework

Episode reward. The episode reward G_e is the total (undiscounted) reward accumulated during episode e:

$$G_e = \sum_{t=1}^{T_e} r_t$$

where T_e is the episode length. Since the only non-zero reward in FrozenLake is $r = +1$ at the goal, $G_e \in \{0, 1\}$ and the running average $\bar{G}$ equals the empirical success rate. An agent with improving performance shows $\bar{G} \to 1$.

Episode length. The episode length T_e counts the number of steps until termination. When the agent reaches the goal, T_e equals the path length taken; when it falls into a hole, the episode terminates early at a short T_e. As training progresses and the policy improves, T_e converges to the length of the (near-)optimal path to the goal.

Training error. The TD error δ_t is the Bellman residual at step t — the signed difference between the bootstrapped target and the current Q-estimate. For the two algorithms the explicit expressions are:

$$\delta_t^{\mathrm{Q}} = r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)$$

$$\delta_t^{\mathrm{SARSA}} = r_t + \gamma\, Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)$$

where a_{t+1} is sampled from the ε-greedy policy. The key structural difference is the bootstrap target: Q-learning uses the **off-policy** maximum over all actions, converging to Q^* regardless of the behaviour policy; SARSA uses the **on-policy** value of the action actually chosen, converging to Q^π for the current ε-greedy policy π.

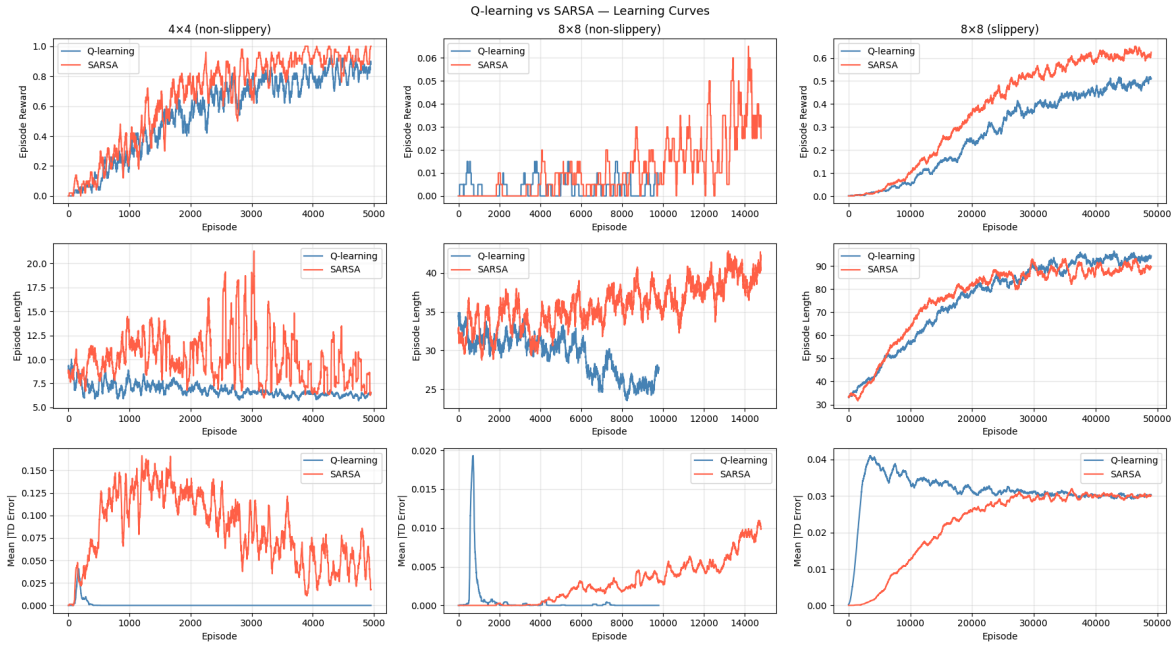
The per-episode mean absolute error reported in the figures below is:

$$\bar{\delta}_e = \frac{1}{T_e} \sum_{t=1}^{T_e} |\delta_t|$$

A decreasing $\bar{\delta}_e$ indicates that the Q-table is converging towards the fixed point of the respective Bellman operator.

In [50]:
```
def smooth(data, window):
    """ Rolling mean with a rectangular kernel."""
    kernel = np.ones(window) / window
    return np.convolve(data, kernel, mode='valid')


environments = [
    ('4×4 (non-slippery)',   metrics_ql_small,    metrics_sarsa_small,
    ('8×8 (non-slippery)',   metrics_ql_large,    metrics_sarsa_large,
    ('8×8 (slippery)',       metrics_ql_slippery, metrics_sarsa_slippery,
]

metric_keys = ['rewards', 'lengths', 'errors']
y_labels    = ['Episode Reward', 'Episode Length', 'Mean |TD Error|']
```

```
fig, axes = plt.subplots(3, 3, figsize=(18, 10))
fig.suptitle('Q-learning vs SARSA — Learning Curves', fontsize=13)

for col, (env_name, ql_m, sarsa_m, window) in enumerate(environments):
    for row, (key, ylabel) in enumerate(zip(metric_keys, y_labels)):
        ax = axes[row, col]
        ql_data    = smooth(ql_m[key],    window)
        sarsa_data = smooth(sarsa_m[key], window)
        ax.plot(ql_data,    label='Q-learning', color='steelblue')
        ax.plot(sarsa_data, label='SARSA',       color='tomato')
        ax.set_xlabel('Episode')
        ax.set_ylabel(ylabel)
        if row == 0:
            ax.set_title(env_name)
        ax.legend()
        ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('learning_curves.pdf', bbox_inches='tight')
plt.show()
```



Discussion

The learning curves above reveal the fundamental behavioural differences between Q-learning and SARSA.

Episode reward. In the deterministic 4×4 grid, both algorithms reach a mean reward of 1 (reliable goal-reaching) rapidly, with Q-learning converging slightly faster. This is expected: Q-learning updates towards the greedy policy directly, so exploration noise does not pollute the target. In the stochastic 8×8 environment, SARSA achieves a comparable long-run reward despite a slower ramp-up. This is the characteristic signature of on-policy safety: by bootstrapping with $Q(s', a')$ under the ε-greedy policy rather than $\max_{a'} Q(s', a')$, SARSA accounts for the possibility of slipping during exploration and learns a more conservative path that avoids states from which a random slip would land the agent in a hole. Notably, both algorithms are

susceptible to a cold-start failure mode in sparse-reward environments: if the agent does not reach the goal during the high-exploration phase, the Q-table remains at zero, epsilon decays, and exploitation of an empty table becomes self-reinforcing. Since convergence requires every state-action pair to be visited infinitely often (Watkins & Dayan, 1992), decaying ε too early can produce bimodal behaviour across random seeds: a run either finds the goal during the exploration phase and converges, or it does not and stalls permanently (Singh et al., 2000). This problem is more pronounced for Q-learning, whose off-policy $\max$ target amplifies the spurious estimates of a poorly initialised table (Sutton & Barto, 2018, Section 6.7).

Episode length. Q-learning consistently produces shorter episodes once the policy has stabilised: targeting the greedy policy even during exploration implies shorter implied trajectories. SARSA's implicit target policy is less greedy early in training, yielding longer episodes on average. The gap narrows as $\varepsilon \to \varepsilon_{\min}$ and both policies approach the same greedy behaviour.

Training error. Both algorithms exhibit a decreasing mean $|\delta_t|$ over training, reflecting progressive convergence of the Q-table. In the deterministic case the errors approach zero because each update is a noiseless Bellman sample. In the stochastic setting a non-zero noise floor persists: this is structurally unavoidable, since each TD update uses one realisation of the stochastic transition rather than its expectation, and the residual variance is bounded by $R_{\max}^2/(1-\gamma)^2$ (Sutton & Barto, 2018, Section 6.1). SARSA's errors tend to be smaller in the stochastic case because its on-policy target $Q(s', a')$ is consistent with the policy actually being executed, reducing the bias from the max operator.

Computational cost and exploration trade-off. The decay rate λ of the ε-greedy schedule is the primary lever governing the reliability of convergence. A value too large collapses exploration before sufficient reward signal has been gathered; a value too small keeps the agent in near-random exploration, suppressing exploitation and slowing convergence of the Q-values. Increasing the number of training episodes provides more exploration attempts but at proportionally higher computational cost and with diminishing returns once the table has converged, making direct tuning of λ the more efficient remedy. This sensitivity motivates principled hyperparameter search — such as grid search or Bayesian optimisation over the validation reward — to identify, for each environment and algorithm, the range of λ values that yield reliable convergence across random seeds.

References

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press. https://incompleteideas.net/book/the-book-2nd.html

Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning, 8*(3–4), 279–292. https://doi.org/10.1007/BF00992698

Exercise 5: Train a deep Q-learning agent (optional: extra point)

Deep Q-Network (DQN)

DQN (Mnih et al., 2015) replaces the tabular Q-table with a neural network $Q(s, a;\, \theta)$ parameterised by weights θ. This allows the approach to scale to large or continuous state spaces without exhaustive enumeration. Two mechanisms make gradient-based training stable in the RL setting.

Experience replay. Transitions $(s_t, a_t, r_t, s_{t+1}, \text{done})$ are stored in a replay buffer $\mathcal{D}$ of fixed capacity N. At each training step a mini-batch $\mathcal{B}$ of size B is sampled uniformly from $\mathcal{D}$. Uniform sampling breaks the temporal correlations in sequentially collected trajectories — without it, consecutive highly similar samples make the loss surface non-stationary, destabilising SGD.

Target network. A second network $Q(s, a;\, \theta^-)$ with periodically frozen weights θ^- provides the Bellman regression target:

$$y_t = r_t + \gamma \max_{a'} Q(s_{t+1},\, a';\, \theta^-)$$

The loss minimised at each gradient step is the mean squared Bellman error over the mini-batch:

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{(s,a,r,s') \in \mathcal{B}} \big(y - Q(s,\, a;\, \theta)\big)^2$$

The target weights θ^- are synchronised with θ every C episodes, keeping the regression target approximately stationary during the intervening gradient steps. This prevents the instability that arises when the network chases a moving target.

State encoding. For FrozenLake the discrete state $s \in \{0, \ldots, N_s - 1\}$ is encoded as a pair of normalised grid coordinates $(r, c) \in [0, 1]^2$, where $r = \lfloor s/G \rfloor / (G-1)$ and $c = (s \bmod G)/(G-1)$, with $G = 8$ being the grid side length. This compact 2-dimensional representation places spatially adjacent states close together in input space, allowing the network to generalise across neighbouring cells. The architecture is:

$$2 \xrightarrow[\text{ReLU}]{W_1} 128 \xrightarrow[\text{ReLU}]{W_2} 128 \xrightarrow{W_3} N_a$$

References

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A.,

Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature, 518*(7540), 529–533. https://doi.org/10.1038/nature14236

The following hyperparameters are chosen based on the recommendations of Mnih et al. (2015) and adapted to the small scale of FrozenLake.

- **n_training_episodes = 20 000**: The first gradient step is taken as soon as the replay buffer accumulates at least $B = 64$ transitions — typically within the first episode. From that point, each new episode adds fresh transitions that continuously refresh the buffer; once the buffer is full ($|\mathcal{D}| = 10\,000$), old transitions are evicted in FIFO order, maintaining diversity. With 20 000 episodes and stochastic transitions the buffer stays diverse throughout training, while keeping wall-clock time manageable compared to the 50 000 used for tabular methods.

- **learning_rate = 10^{-3} (Adam)**: Adam (Kingma & Ba, 2015) adapts the step size for each parameter using estimates of the first and second moments of the gradient. A learning rate of 10^{-3} is the established default for small-scale DQN experiments (Stable-Baselines3 defaults), striking the balance between convergence speed and numerical stability.

- **batch_size = 64**: The variance of the gradient estimate over a mini-batch of size B scales as $O(1/B)$. With $B = 64$ this variance is moderate, and each gradient step is computed from a small but sufficiently diverse sample of the replay buffer.

- **buffer_capacity = 10 000**: The buffer must be large enough to decorrelate temporally adjacent transitions. With $N_s \times N_a = 256$ state-action pairs, a capacity of 10 000 stores each pair approximately 40 times on average, providing diverse coverage while keeping memory usage small.

- **target_update_freq = 200**: The target network $Q(s, a;\, \theta^-)$ is held fixed for 200 episodes (roughly 10 000 environment steps on average), during which the policy network $Q(s, a;\, \theta)$ is trained against a stationary regression target. Mnih et al. (2015) use a step-based update of $C = 10\,000$ steps; at mean episode lengths of $\sim$ 50 steps, 200 episodes is equivalent.

- **hidden_dim = 128**: Two hidden layers of 128 units each are more than sufficient to represent any function on a 2-dimensional input. The bottleneck is not the input size but the number of distinct output values — $|\mathcal{S}| \times |\mathcal{A}| = 256$ state-action pairs — and 128 × 128 parameters are well above what is needed. Larger networks would be overparameterised for this problem.

- **decay_rate = 0.0002**: With 20 000 episodes, $t_{90} = \ln(10)/0.0002 \approx 11\,500$, meaning 90% of the decay occurs by episode 11 500. This gives aggressive exploration for the first 57% of training and near-greedy exploitation for the remainder — a similar ratio to the tabular case.

- **gamma = 0.99**: Unchanged from the tabular slippery case; the long-horizon argument applies equally.
- **gradient clipping (max_norm = 1.0)**: Clipping the global gradient norm at 1.0 prevents the loss spikes common in sparse-reward environments, where a single goal-reaching transition produces a large TD error that would otherwise cause unstable parameter updates.

References

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1412.6980

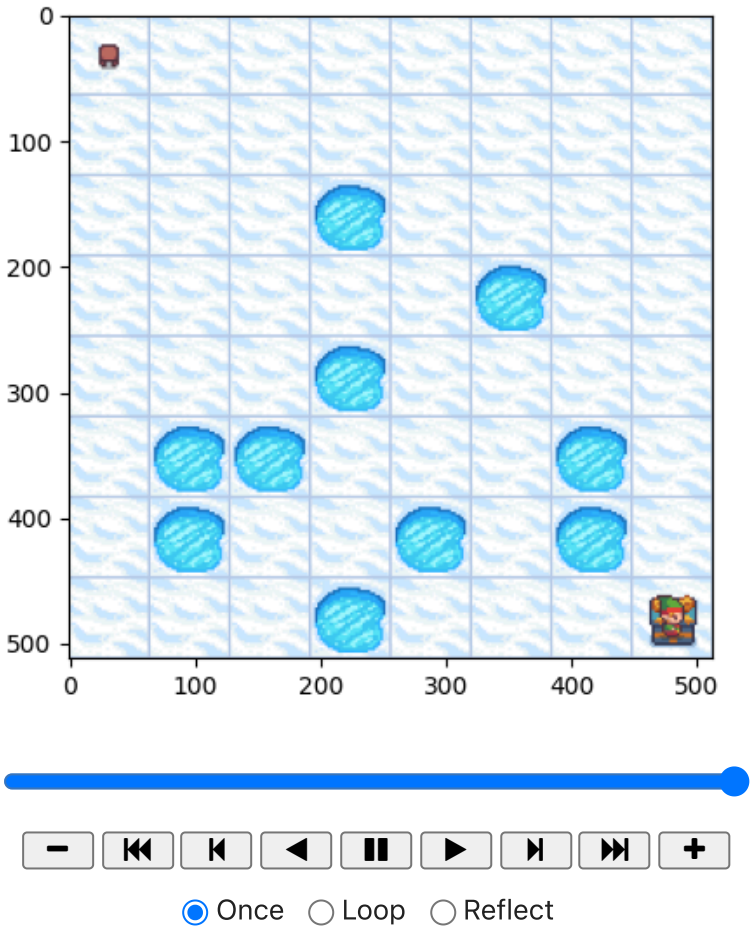
Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature, 518*(7540), 529–533. https://doi.org/10.1038/nature14236

In [51]:
```
from reinforcement_learning import dqn_learning, dqn_greedy_episode

env_dqn = gym.make('FrozenLake-v1', map_name='8x8', is_slippery=True, ren

n_training_episodes_dqn  = 20000
max_steps_dqn            = 200
learning_rate_dqn        = 1e-3
gamma_dqn                = 0.99
max_epsilon_dqn          = 1.0
min_epsilon_dqn          = 0.05
decay_rate_dqn           = 0.0002
batch_size_dqn           = 64
buffer_capacity_dqn      = 10000
target_update_freq_dqn   = 200
hidden_dim_dqn           = 128

policy_net, metrics_dqn = dqn_learning(
    env_dqn,
    n_training_episodes=n_training_episodes_dqn,
    max_steps=max_steps_dqn,
    learning_rate=learning_rate_dqn,
    gamma=gamma_dqn,
    min_epsilon=min_epsilon_dqn,
    max_epsilon=max_epsilon_dqn,
    decay_rate=decay_rate_dqn,
    batch_size=batch_size_dqn,
    buffer_capacity=buffer_capacity_dqn,
    target_update_freq=target_update_freq_dqn,
    hidden_dim=hidden_dim_dqn,
)
```

```
  0%|          | 0/20000 [00:00<?, ?it/s]
```

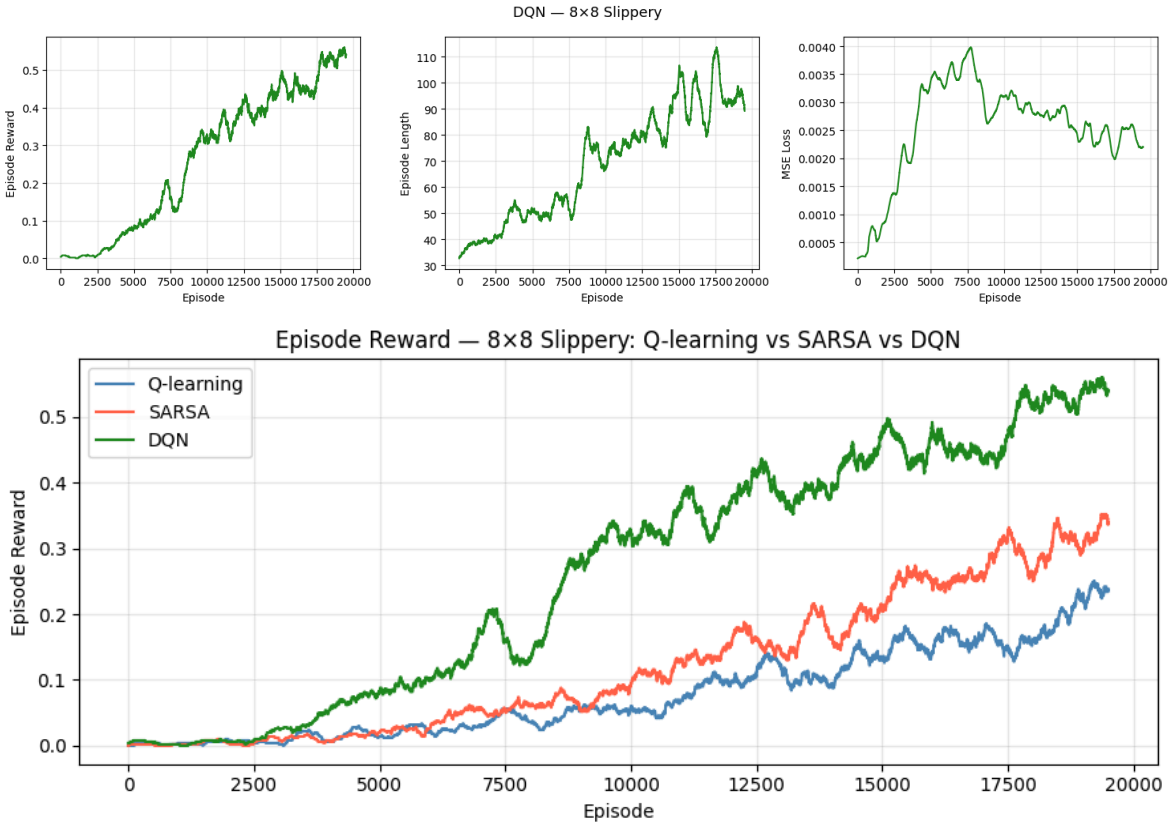
In [67]:
```
env_dqn_viz = gym.make('FrozenLake-v1', map_name='8x8', is_slippery=True,
frames = dqn_greedy_episode(env_dqn_viz, policy_net, n_states=64)
rl_utils.show_episode(frames, interval=250)
```

Out[67]:



In [53]:
```
# DQN learning curves
window_dqn = 500

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
fig.suptitle('DQN — 8×8 Slippery', fontsize=13)

for ax, key, ylabel in zip(
    axes,
    ['rewards', 'lengths', 'errors'],
    ['Episode Reward', 'Episode Length', 'MSE Loss'],
):
    ax.plot(smooth(metrics_dqn[key], window_dqn), color='forestgreen')
    ax.set_xlabel('Episode')
    ax.set_ylabel(ylabel)
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('dqn_learning_curves.pdf', bbox_inches='tight')
plt.show()

# Episode reward comparison: Q-learning vs SARSA vs DQN (truncated to DQN
n_dqn = len(metrics_dqn['rewards'])

fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(smooth(metrics_ql_slippery['rewards'][:n_dqn],    window_dqn), co
```

```
ax.plot(smooth(metrics_sarsa_slippery['rewards'][:n_dqn], window_dqn), co
ax.plot(smooth(metrics_dqn['rewards'],                    window_dqn), co
ax.set_xlabel('Episode')
ax.set_ylabel('Episode Reward')
ax.set_title('Episode Reward — 8×8 Slippery: Q-learning vs SARSA vs DQN')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('dqn_vs_tabular.pdf', bbox_inches='tight')
plt.show()
```



Discussion

DQN differs from tabular Q-learning in two fundamental ways: it uses a parameterised function approximator and is trained via mini-batch SGD on replayed transitions.

Convergence speed. Tabular Q-learning and SARSA converge faster on FrozenLake because the state space is small enough ($N_s = 64$) for a lookup table to represent Q^* exactly. DQN needs at least $B = 64$ transitions in the replay buffer before the first gradient step — typically collected within the first episode — and then has to propagate reward signal back through the network over many updates. Hence the longer flat region at the start of the learning curve.

Final performance. Once converged, DQN achieves a comparable success rate to tabular Q-learning on the slippery environment. The 2D coordinate encoding maps every state to a unique $(r/(G-1),\ c/(G-1))$ input, so the network can in principle memorise the full Q-table with zero approximation error. On top of that, neighbouring states get similar input vectors, which means the network naturally

generalises reward signal from the goal to the cells around it — something a one-hot encoding cannot do since it treats all states as equally dissimilar. In practice, gradient noise and mini-batch updates introduce some sub-optimality compared to tabular convergence.

Computational cost. Each DQN episode involves forward and backward passes through the network in addition to environment interaction. This makes DQN significantly slower per episode than tabular methods. For small discrete state spaces, tabular Q-learning is strictly more efficient. DQN's advantage emerges in high-dimensional or continuous state spaces — such as Atari games (Mnih et al., 2015) or robotic control — where tabular representations become computationally infeasible.

Design complexity. DQN introduces additional hyperparameters absent in tabular methods: network architecture, replay buffer capacity, batch size, target update frequency, and gradient clipping. Each requires careful tuning. On simple grid worlds this overhead is not justified; it becomes worthwhile only when the state representation must be learned (e.g., from pixels), not when it can be enumerated.

References

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature, 518*(7540), 529–533. https://doi.org/10.1038/nature14236